Improved Caesar, Playfair and Hill

Proposal of a stronger version of each algorithm, against brute force and the classic attacks — to be validated before it goes into the project

0. Why the original versions fall

Even with the fixes already in the TP, each algorithm keeps a structural weakness that an attacker can exploit:

Algorithm	Attack	Why it works
Caesar	Brute force	Only 62 useful keys: trying all of them takes a fraction of a second.
Caesar	Frequency analysis	The same letter is always encrypted the same way, so the most frequent symbol of the ciphertext is probably “e” or space.
Playfair	Dictionary attack	The key is a word; trying every word of a dictionary is fast.
Playfair	Pair-frequency analysis	The same pair always gives the same result (and AB→CD implies BA→DC).
Hill	Brute force	A 2×2 key mod 26 has only 157 248 invertible matrices.
Hill	Known plaintext	Hill is linear: knowing n clear/encrypted blocks gives a system of equations whose solution is the key.
All three	Same message, same result	Encrypting twice the same message gives the same ciphertext: an observer sees when a message repeats.
All three	Modification	Nothing tells B that the ciphertext was changed on the way: B decrypts garbage without knowing it.

The new versions keep the principle of each algorithm (a shift, a grid with the 3 Playfair rules, a matrix product), but remove these weaknesses with **building blocks shared by all three**, described in section 1.

1. Common building blocks

1.1 A larger alphabet: 99 symbols

All three algorithms work on the same alphabet: the 95 printable ASCII characters (space, punctuation, digits, upper- and lowercase letters) plus the 4 French letters é è à ç. Nothing is dropped any more: B receives exactly the message A typed, with its spaces, case and punctuation.

```
index  0 = space    1 = !    …    16 = 0 … 25 = 9    …    33 = A … 58 = Z
index 65 = a … 90 = z    …    95 = é    96 = è    97 = à    98 = ç
```

1.2 A passphrase instead of a small key

The secret shared by A and B is now a **passphrase** (for example `Ssad-Tp1-2026!`). The number n, the Playfair grid and the Hill matrix are no longer chosen by hand: they are **derived** from the passphrase. This links directly to Part 3 of the TP (password policy).

1.3 A new random value for every message (nonce)

For every message, A draws **16 random bytes** (the nonce, “number used once”) and sends them in clear with the message. The nonce is not secret; its role is to make every message use different internal keys. Consequence: **the same message encrypted twice gives two completely different ciphertexts.**

1.4 Slow key derivation (against brute force)

From the passphrase and the nonce, A and B compute two 32-byte keys with **PBKDF2-HMAC-SHA256** and **200 000 iterations**:

```
(K_enc , K_mac) = PBKDF2-HMAC-SHA256(passphrase, nonce, 200 000 iterations, 64 bytes)
```

The 200 000 iterations are there on purpose: one derivation takes **0.137 s** (measured). For A and B it is invisible, but an attacker pays this price **for every passphrase they try**. See section 6 for the numbers.

1.5 A keystream of secret numbers

From K_enc, both sides generate the same sequence of secret bytes:

```
block 0 = HMAC-SHA256(K_enc, 0)   block 1 = HMAC-SHA256(K_enc, 1)   …   (32 bytes per block)
```

Each algorithm reads numbers from this stream (a shift for Caesar, a grid for Playfair, a matrix and masks for Hill). To get a number uniformly between 0 and m−1, a byte is kept only if it is below the largest multiple of m that fits in 256 (for m = 99: bytes ≥ 198 are thrown away). Without this, some values would come out more often than others.

1.6 An integrity tag (against modification)

A computes a 128-bit tag over the algorithm name, the nonce and the ciphertext:

```
tag = HMAC-SHA256(K_mac, algorithm ‖ nonce ‖ ciphertext), first 16 bytes
```

B recomputes the tag **before** decrypting. If one single character was changed, or if the passphrase is wrong, the tags differ and B refuses: `Message altéré ou mauvaise clé : déchiffrement refusé`. This also covers Part 4 of the TP (integrity).

1.7 What A sends to B

```
nonce (32 hex) : ciphertext : tag (32 hex)

3f9a0c71d24e8b56a1f03c97e62d4b18:-(^2ZR3 &=E~]@+:cb7df4fe6948aba0240acf3bb2dd4da3
```

This adds 66 characters to every message. B reads the nonce from the first 32 characters and the tag from the last 32, so a “:” inside the ciphertext is not a problem.

```
ENTITY A
  nonce        ← 16 random bytes
  K_enc, K_mac ← PBKDF2(passphrase, nonce)
  ciphertext   ← algorithm(message, keystream(K_enc))
  tag          ← HMAC(K_mac, algorithm ‖ nonce ‖ ciphertext)

        ————————  nonce : ciphertext : tag  ————————▶

ENTITY B
  K_enc, K_mac ← PBKDF2(passphrase, nonce)
  check the tag   →  refuse if it is different
  message      ← inverse algorithm(ciphertext, keystream(K_enc))
```

2. Caesar with variable shifts

Idea

The problem of Caesar is that **one** shift is used for the whole message. In the new version, **every character gets its own secret shift** s_i, read from the keystream. The n and the direction from the TP statement are kept, but the security comes from the s_i.

Entity A encrypts

1. Draw a nonce, derive K_enc and K_mac (section 1).
2. For each character at position i: read s_i (0 … 98) from the keystream.
3. Encrypt: `c = ALPHABET[(index + n + s_i) mod 99]` (Gauche → Droite), or `− (n + s_i)` (Droite → Gauche).
4. Compute the tag and send `nonce:ciphertext:tag`.

Example

Message `Salut B, rdv 9h`, passphrase `Ssad-Tp1-2026!`, n = 3, Gauche → Droite, nonce `3f9a0c71…4b18` (fixed here for the example; random in practice).

	S	**a**	**l**	**u**	**t**	**␣**	**B**	**,**	**␣**	**r**	**d**	**v**	**␣**	**9**	**h**
index	51	65	76	85	84	0	34	12	0	82	68	86	0	25	72
s_i	58	39	82	29	70	47	81	84	3	43	65	5	58	4	35
+3+s_i mod 99	13	8	62	18	58	50	19	0	6	29	37	94	61	32	11
result	-	(	^	2	Z	R	3	␣	&	=	E	~	]	@	+

```
A sends:  3f9a0c71d24e8b56a1f03c97e62d4b18:-(^2ZR3 &=E~]@+:cb7df4fe6948aba0240acf3bb2dd4da3
```

Check of the first character: S has index 51, s_0 = 58, so 51 + 3 + 58 = 112 mod 99 = 13 → `-`.

Entity B decrypts

B reads the nonce, derives the same K_enc and K_mac, **checks the tag**, then regenerates the same s_i and subtracts them: `ALPHABET[(index − n − s_i) mod 99]`. B gets back exactly `Salut B, rdv 9h`.

Why it resists

Attack	Result with the new version
Brute force	Trying n is useless: the s_i depend on the passphrase, and each passphrase tried costs 0.137 s (section 6).
Frequency analysis	A repeated letter gives different symbols. aaaaaaaaaa gives `;(SàG0RUg,`: no frequency to analyse.
Same message twice	With another nonce, the same message gives `>3V:.&ké?]E%}mS`.
Known plaintext	Knowing a clear/encrypted pair reveals the s_i of that message only. The next shifts and the other messages stay unknown, because they come from HMAC.
Modification	Changing the first encrypted character `-` into X → tag differs → refused.
Wrong passphrase	`ssad-tp1-2026!` (one capital letter missing) → refused.

3. Playfair with a new grid for every pair

Idea

Playfair falls because **one** grid encrypts the whole message, and because this grid comes from a word. In the new version:

- the grid is **10×10 = 100 symbols**: the 99 symbols of the alphabet + a filler ¤ that can never appear in a message;
- the grid is no longer “keyword + rest of the alphabet”, it is a **random shuffle** (Fisher–Yates) driven by the keystream;
- **every pair is encrypted with its own grid**: the grid is shuffled again before each pair.

The 3 Playfair rules (same row → right, same column → down, rectangle → swap the columns) are unchanged.

Entity A prepares the message

Message `Ballon 2026`. Nothing is removed (space and case are kept). Cut into pairs; a doubled letter or a lone last letter gets the filler ¤:

```
Ba   l¤   lo   n␣   20   26
```

Because ¤ is not in the alphabet of messages, B can remove it without any ambiguity. The old problem “AXA comes back as AA” disappears.

Entity A encrypts

Grid 1, used only for the first pair Ba. B (blue) is at (0,4), a at (8,9): rectangle, so each keeps its row and takes the other’s column (green) → +ç.

A	<	|	8	**B**	n	6	à	!	**+**
,	x	9	m	)	.	p	H	~	M
'	`	U	l	y	I	é	-	^	1
7	V	F	:	s	$	>	i	Q	C
4	{	W	L	z	Z	h	è	(	b
j	X	\	3	q	2	&	␣	"	]
}	E	[	J	#	c	¤	T	_	/
5	;	R	@	w	o	u	=	0	g
k	N	d	D	**ç**	P	0	e	%	**a**
K	*	t	v	S	f	?	r	G	Y

Pair	Grid used	Positions (row, col)	Rule	Result
Ba	grid 1	B(0,4) a(8,9)	rectangle	**+ç**
l¤	grid 2	l(5,6) ¤(0,8)	rectangle	**é!**
lo	grid 3	l(8,0) o(1,6)	rectangle	**0è**
n␣	grid 4	n(4,5) ␣(1,6)	rectangle	**Zu**
20	grid 5	2(0,3) 0(3,8)	rectangle	**ç>**
26	grid 6	2(4,2) 6(5,4)	rectangle	**P<**

```
A sends:  3f9a0c71d24e8b56a1f03c97e62d4b18:+çé!0èZuç>P<:04690a15b8a53b6b7f76b500c598ef2b
```

Entity B decrypts

B derives the same keys, checks the tag, then regenerates grid 1, grid 2, … in the same order. On each grid, B applies the reverse rules (same row → left, same column → up, rectangle → same rule), then removes every ¤. B gets back exactly `Ballon 2026`.

Why it resists

Attack	Result with the new version
Dictionary / brute force	The grid comes from the passphrase through PBKDF2: each passphrase tried costs 0.137 s. The number of possible grids is 100! ≈ 9.3×10^{157}.
Pair-frequency analysis	Each pair uses a different grid, so the same pair gives different results; the relation AB→CD ⇒ BA→DC no longer holds.
Known plaintext	A clear/encrypted pair gives information only about the grid of that pair, which is never used again.
Same message twice	Another nonce gives other grids: `(@JgR4s>52Jk`.
Lost information	Case, spaces, digits, punctuation and é è à ç are kept; filler removal is unambiguous.
Modification	Detected by the tag, as for Caesar.

4. Masked Hill

Idea

Hill falls for two reasons: the key space of a small matrix is small, and Hill is **linear** (known-plaintext attack). In the new version:

- the matrix is **4×4 modulo 99**, generated from the keystream (new matrix for every message, thanks to the nonce); a matrix is kept only if gcd(det, 99) = 1, so B can always invert it;
- every block gets a **secret mask vector** V_i (4 new numbers from the keystream): `C_i = K·P_i + V_i (mod 99)`;
- padding is **always** added and says its own length: p symbols of value p ($1 \le p \le 4$). B reads the last value and removes exactly p symbols. The old “HELLOX” problem disappears.

Entity A encrypts

Message `Hello` → values (H=40, e=69, l=76, o=79). 5 values, so p = 3 padding values of 3 are added: 8 values = 2 blocks of 4.

Matrix generated for this message (det = 82, gcd(82, 99) = 1 → invertible):

```
K =
    | 52  46  27  29 |
    | 26  48  45   7 |
    | 24  72  64  56 |
    | 33  21  57  52 |
```

Block	Values P	K·P mod 99	Mask V	C = K·P + V mod 99	Result
Hell	[40, 69, 76, 76]	[6, 87, 0, 64]	[34, 71, 0, 77]	[40, 59, 0, 42]	**H[␣J**
o + padding	[79, 3, 3, 3]	[58, 77, 96, 27]	[58, 2, 17, 6]	[17, 79, 14, 33]	**1o.A**

```
A sends:  3f9a0c71d24e8b56a1f03c97e62d4b18:H[ J1o.A:1c5357064450813755b5a5698d6a04ad
```

Entity B decrypts

B derives the same keys, checks the tag, regenerates the same K and the same masks, computes K^{-1} mod 99 (adjugate × inverse of det mod 99), then for each block: `P_i = K⁻¹ · (C_i − V_i) (mod 99)`.

```
K⁻¹ =
    | 36  95  24  46 |
    | 34  40  81  75 |
    |  6  54  88  85 |
    | 33  30   6  13 |
```

B gets the 8 values back, reads the last one (3), removes 3 values, and obtains exactly `Hello`.

Why it resists

Attack	Result with the new version
Brute force	K is not typed by the user: it comes from the passphrase through PBKDF2 (0.137 s per try). Even ignoring that, 4×4 matrices mod 99 are ≈ 99^{16} ≈ 8.5 × 10^{31}.
Known plaintext	Each block adds 4 equations but also 4 new unknowns (V_i): the system never has a single solution, so K cannot be computed. And K changes with every message.
Same message twice	Another nonce gives another K and other masks: `2AomUY:;`.
Repeated letters	The two “l” of Hello are in the same block but give different symbols; changing one letter changes the 4 symbols of its block (diffusion).
Lost information / padding	Case, spaces and punctuation are kept; padding removal is exact.
Modification	Detected by the tag.

5. Old versions vs new versions

	Old Caesar	New Caesar	Old Playfair	New Playfair	Old Hill	New Hill
Secret	n (62 values)	passphrase	keyword	passphrase	2×2 / 3×3 matrix	passphrase
Brute force	instant	0.137 s / try	fast (dictionary)	0.137 s / try	fast (2×2)	0.137 s / try
Frequency analysis	works	fails	works on pairs	fails	partly	fails
Known plaintext	gives the key	gives nothing else	gives grid info	gives nothing else	gives the key	gives nothing
Same message → same cipher	yes	no	yes	no	yes	no
Modification detected	no	yes	no	yes	no	yes
Text kept exactly	almost	yes	no	yes	no	yes

6. Cost of a brute-force attack

With the new versions, attacking means trying passphrases, and each try costs one PBKDF2 derivation: **0.137 s** on one processor core (measured), that is about 7 tries per second.

Passphrase	Number of possibilities	Time at 7 tries/s (1 core)	Time at 1 000 000 tries/s (very large attacker)
Old Caesar key	62	—	instant (no derivation)
Word from a 1 000 000-word list	10^6	≈ 38 hours	≈ 1 second
8 characters, 94 symbols	≈ 6.1 × 10^{15}	≈ 27 million years	≈ 190 years
12 characters, 94 symbols	≈ 4.8 × 10^{23}	> 10^{15} years	≈ 1.5 × 10^{10} years

Conclusion: the slow derivation makes brute force on a random passphrase hopeless, but **a dictionary word is still found**. So the interface must impose a passphrase policy: at least 12

characters, with upper- and lowercase letters, digits and symbols, and not a dictionary word. This is exactly the subject of Part 3 of the TP.

7. Honest limits

- **Security rests on the passphrase.** A weak passphrase breaks all three algorithms, whatever the algorithm.
- **The length of the message is visible** (exactly for Caesar, rounded to the pair or the block for Playfair and Hill).
- **Playfair:** by construction of the rules, a symbol is never encrypted into itself at the same position. This tiny bias is harmless here because each grid is used once, but it exists.
- **What really protects:** the new versions owe their strength to standard primitives (PBKDF2, HMAC-SHA256, random nonce). Caesar, Playfair and Hill keep their structure and their names, but on their own they would still be weak. For a real application, the right choice is a modern algorithm such as AES-GCM or ChaCha20-Poly1305; these versions are an educational answer to the TP.
- **Cost:** 66 extra characters per message, and about 0.14 s per encryption or decryption (the derivation).

8. What will change in the project (after your approval)

Where	Change
`chiffrement.py`	Add the shared part (alphabet of 99 symbols, PBKDF2, keystream, tag, message format) and the 3 new pairs of functions. The old functions stay, so the old and new versions can be compared in the report.
Interface — Chiffrement page	A choice “Version : Classique / Renforcée”. In the reinforced version, the key becomes a passphrase field (hidden, with “Afficher”); Caesar keeps n and the direction; the Playfair grid preview and the Hill matrix cells are replaced by the passphrase.
Interface — messages	Clear errors: unsupported character, message modified or wrong passphrase, invalid format.
Tests	The prototype already passes, for each algorithm, 150 random messages: decryption gives back the message, the same message gives two different ciphertexts, a modified message and a wrong passphrase are refused.